Circle Dodge Process Work

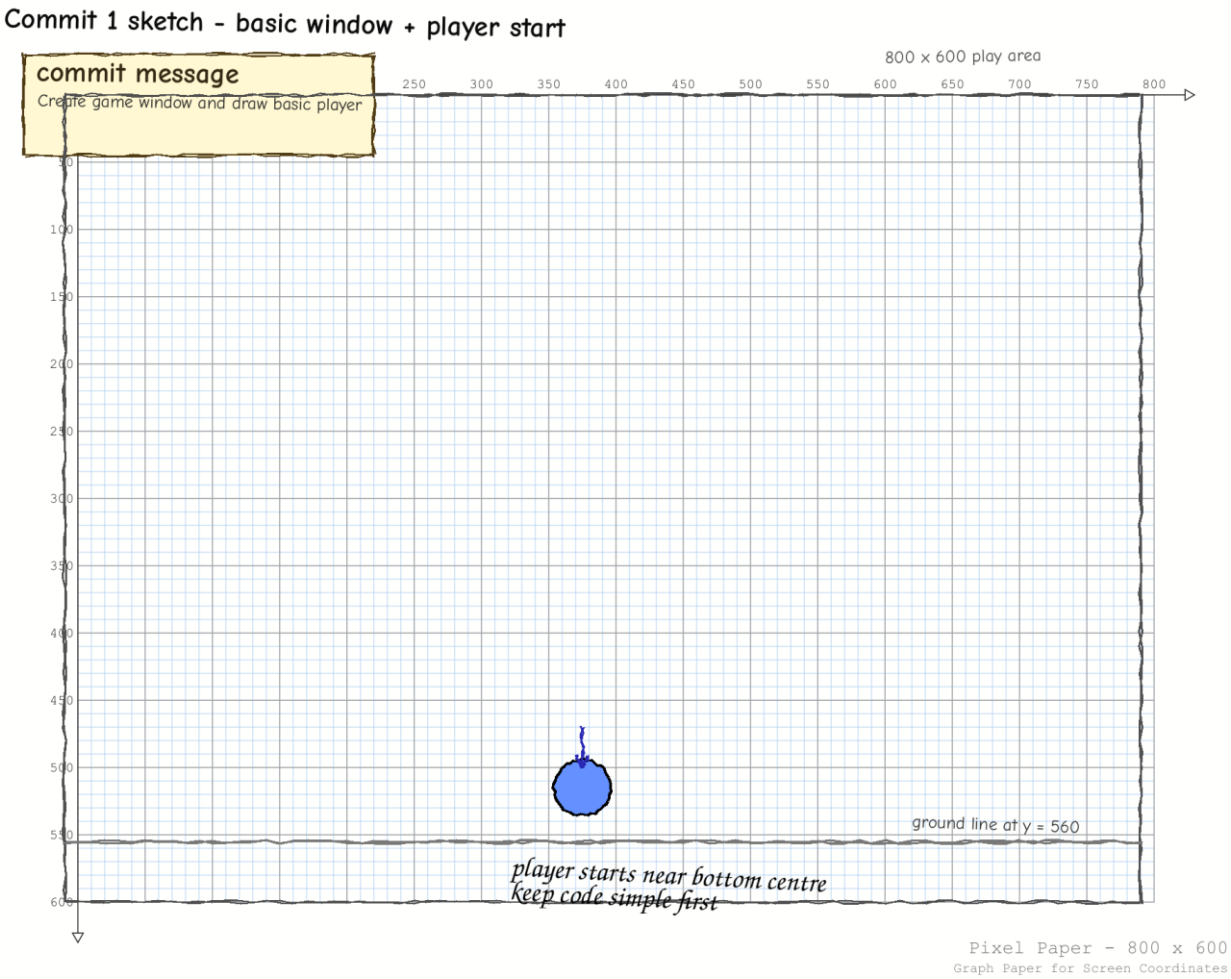
1. Planning Overview

- Player class controls movement and screen boundaries.
- Falling Enemy class represents one red enemy object; an array of 10 enemies satisfies the class-array requirement.
- Star Collectible class adds score progression and makes the game repayable.
- The final version removes the 5-point win cap and instead runs infinitely until the player touches an enemy.
- Each sketch includes coordinate placement, gameplay notes, and the change represented by that commit.

2. Commit 1 - Initial Window and Player

Commit	Create game window and draw basic player

The first sketch only plans the play space, ground line, and player starting position. This is a safe first commit because it proves the template window works before adding gameplay logic.



- The blue circle is the player start position near the bottom centre of the screen.
- The grey horizontal line marks the ground / lower movement boundary.
- At this point there are no enemies yet; the focus is just getting a visible object on screen.

Problem

When I first added movement, the player circle could move off the screen. This made the game look unfinished and made it possible for the player to disappear partially or completely.

Problem-Solving Process

I first tested the movement controls with both WASD and arrow keys to make sure input worked. After confirming movement worked, I noticed the player could go beyond the left, right, top, and bottom edges of the window. I solved this by adding boundary checks after updating the player position. I compared the player position to the window size and used the player radius so the full circle stayed

visible. I also limited the bottom movement to stay above the ground line so the player remained inside the play area.

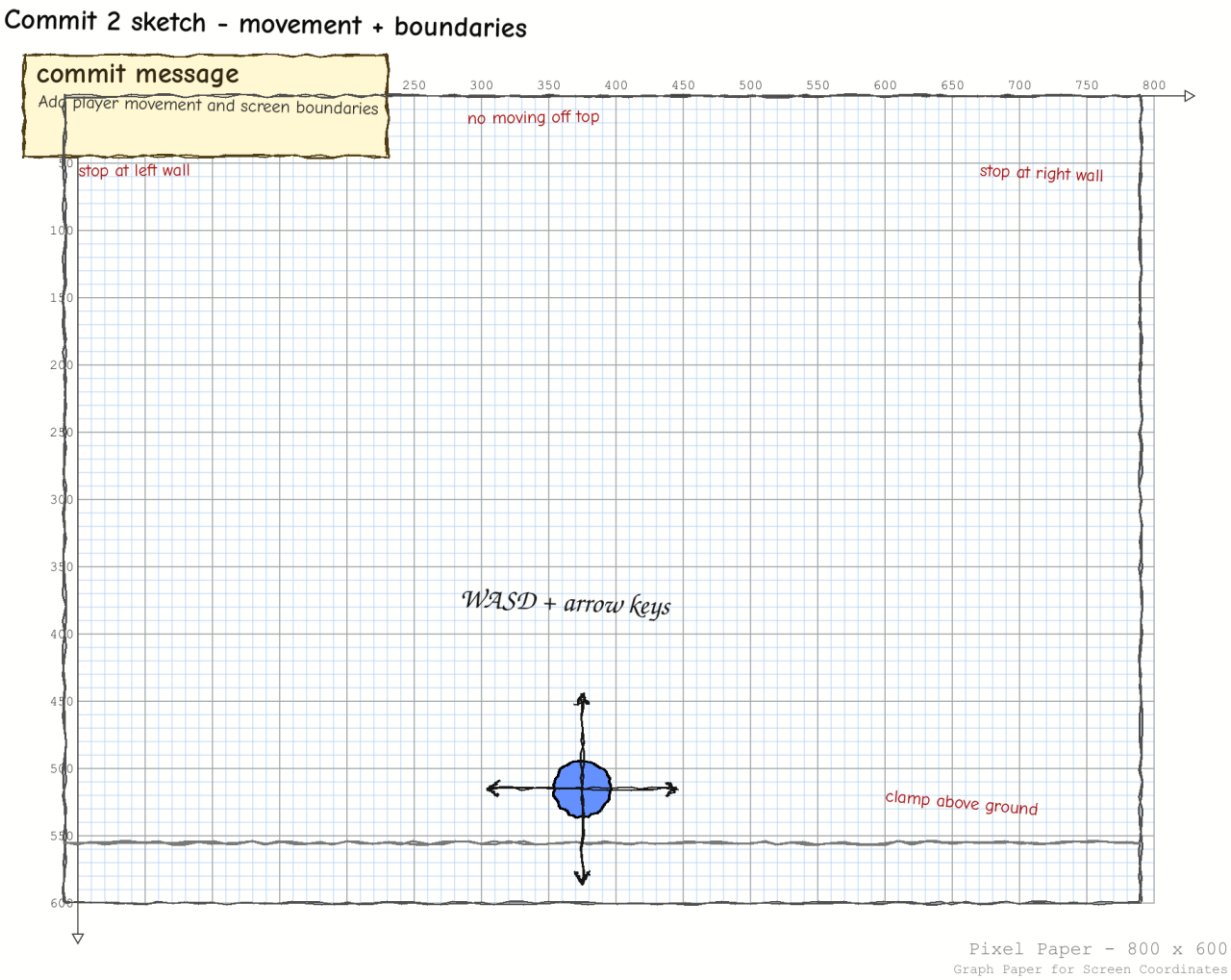
Final Decision

I kept simple screen-boundary checks because they were easy to understand, worked consistently, and matched the course concepts.

3. Commit 2 - Movement and Boundaries

Commit	Add player movement and screen boundaries

The second sketch adds keyboard movement and clamping rules. I planned the player to move with WASD and Arrow Keys while staying inside the 800 x 600 play area.



- Arrows show intended movement directions before coding the Update() movement logic.
- Handwritten notes mark the left wall, right wall, top limit, and ground limit.
- This step directly supports the rubric item for input and motion behavior.

Problem

When I first added enemies, they did not look evenly organized and some spawned too close together. This made the layout messy and harder to test.

Problem-Solving Process

My first idea was to place all enemies completely randomly, but that caused overlapping and inconsistent spacing at the start of the game. I changed the setup so each enemy started with a calculated X position using a pattern across the screen, and a staggered Y position above the top of the window. This made the enemies enter the screen one after another instead of all at once. After that, I still kept random repositioning when enemies reset so the gameplay would not feel too repetitive.

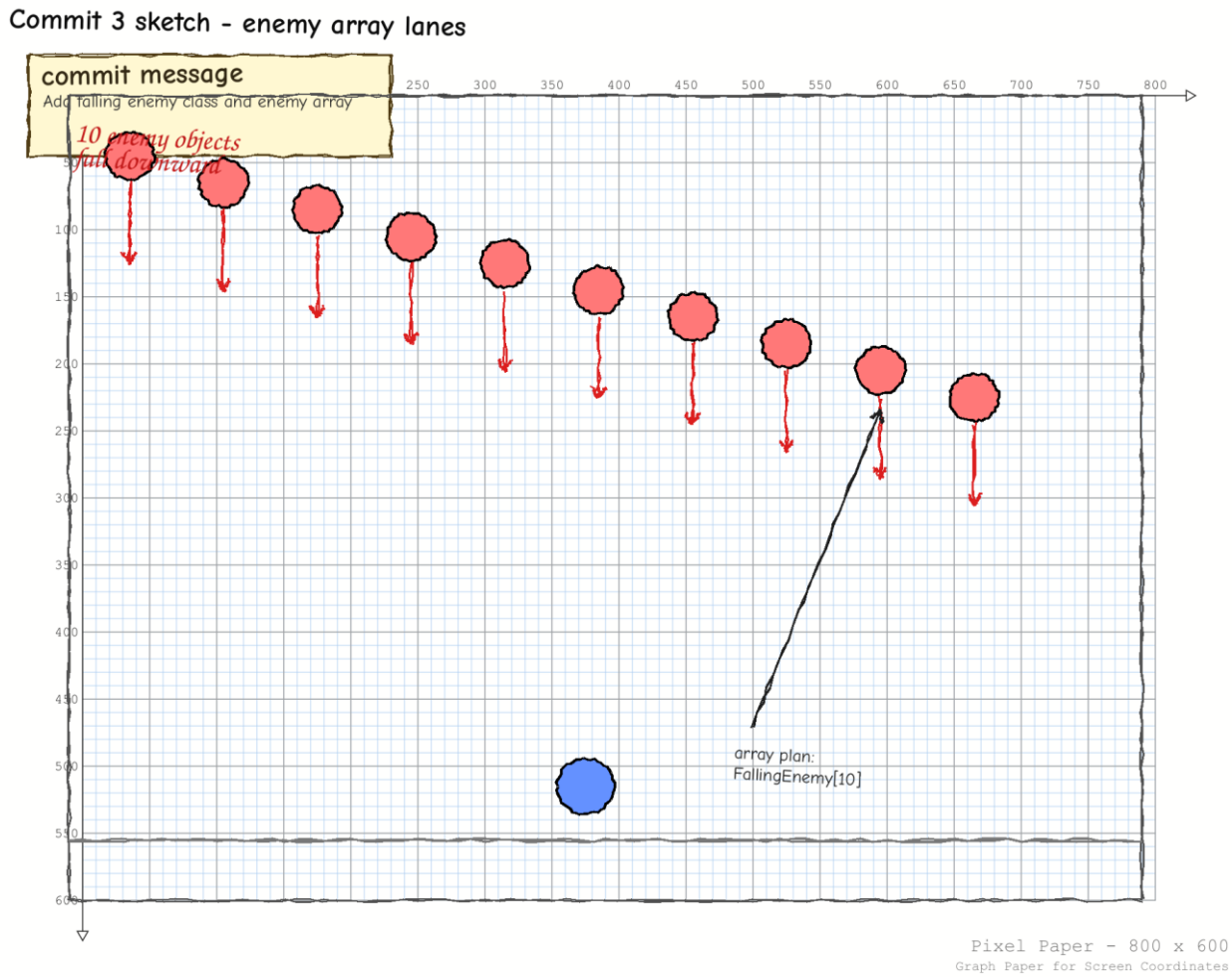
Final Decision

I used structured starting positions and randomized respawn positions. This gave me better testing at the start while still making the gameplay feel different over time.

4. Commit 3 - Enemy Array Planning

Commit	Add falling enemy class and enemy array

The third sketch is where the class-array requirement is clearly planned. Each red circle represents one FallingEnemy object and the row of ten enemies maps directly to FallingEnemy[10] in code.



- The repeated downward arrows show that every enemy object shares the same falling behaviour.
- The note box on the page identifies the commit and the intention to build the enemy array next.
- This sketch supports both the user-defined class requirement and the array-of-class-type requirement.

Problem

The game had movement and enemies, but it did not yet have a clear goal. It felt more like a movement test than a complete game

Problem-Solving Process

I thought about several ways to make the game feel more complete. One option was a survival timer, and another option was a collectible object. I chose to add a collectible star because it gave the player a reason to move around the screen and created risk-reward gameplay. After adding the star, I needed collision detection for both enemies and collectibles. I reused the circle collision logic so the same method could work for both object types. Then I added score tracking and simple win/lose conditions so the game had a clear outcome.

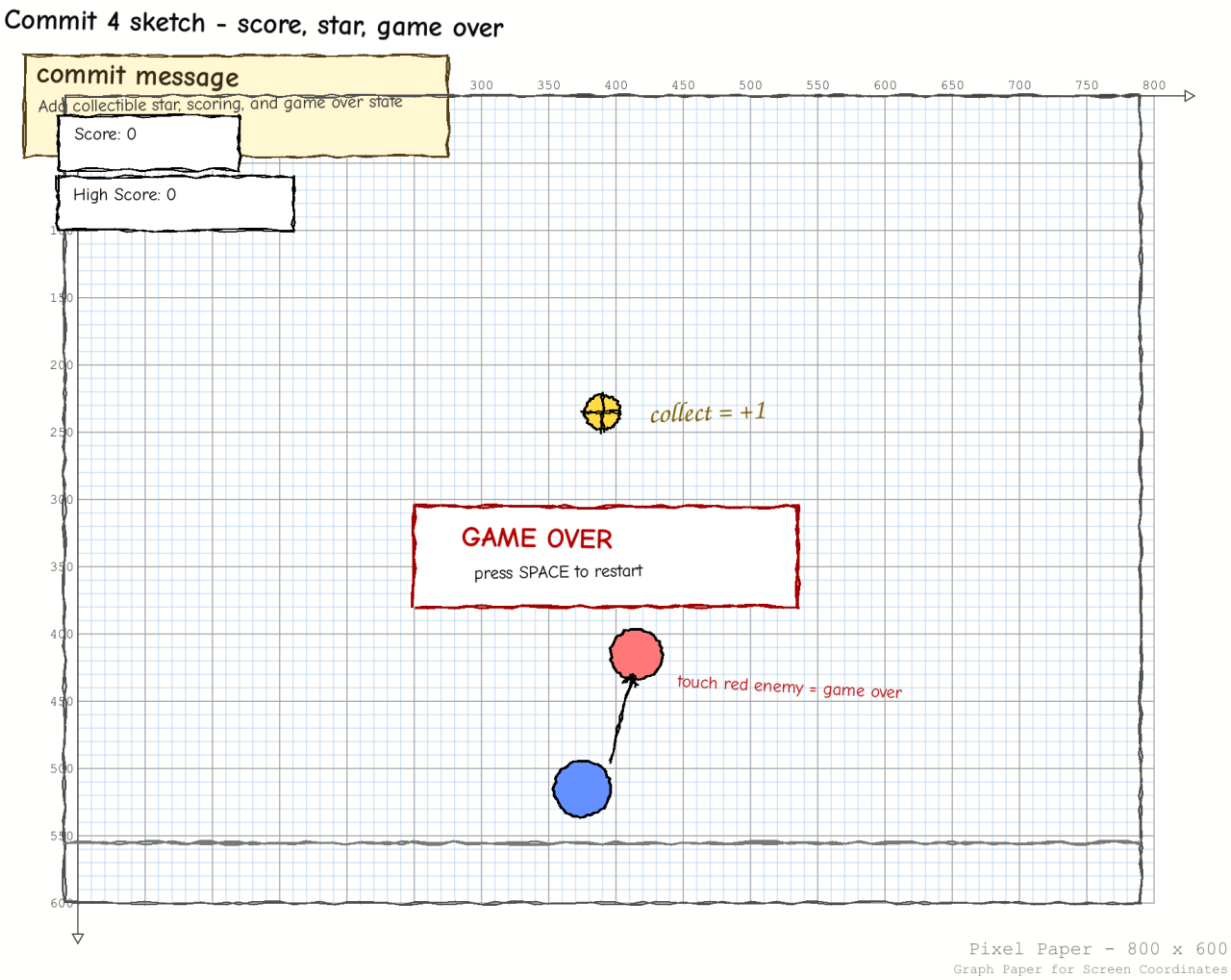
Final Decision

I chose a collectible-based score system because it was simple to implement, easy to explain in process work, and matched the visual style of the game.

5. Commit 4 - Scoring, Collision, and Restart

Commit	Add collectible star, scoring, and game over state

The fourth sketch adds the core interaction rules that make the game playable: collect a star for score, lose on collision with a red enemy, and restart after game over.



- The yellow star stands for the collectible object that increases score by +1.
- The boxed GAME OVER note shows the restart screen flow triggered after a collision.
- This step is the main logic commit where collision detection and game states are connected.

Problem

Once the gameplay worked, the screen looked too plain and some parts of the code were harder to read quickly. The project met the minimum requirements, but it did not yet look polished for submission.

Problem-Solving Process

I reviewed the game as if I were submitting it and asked what parts looked unfinished. The background was very empty, the collectible was too simple, and the instructions were not clear enough for a first-time player. I added background shapes, clearer UI text, and a more visible collectible design so important objects stood out better. I also went back through the code and added comments to explain key variables, methods, and logic. This made the project easier to understand and easier to present as a finished assignment.

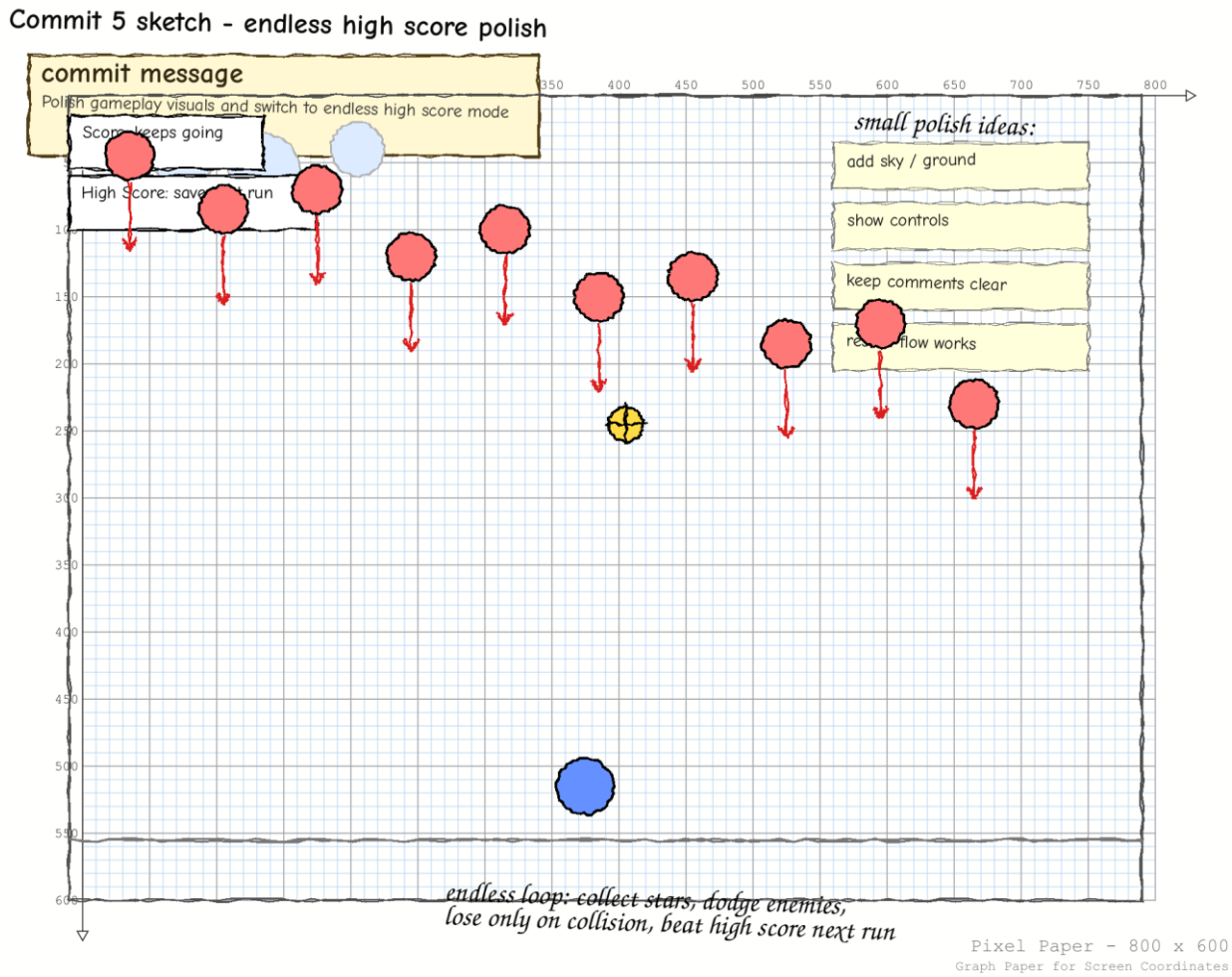
Final Decision

I kept the visuals simple instead of over-designing them, because the assignment focused more on programming structure and process work than on advanced art.

6. Commit 5 - Endless High Score Polish

Commit	Polish gameplay visuals and switch to endless high score mode

The final sketch shows the polished concept: an endless loop with high-score tracking instead of a short 5-point win condition. Background notes also mark extra polish such as controls text, simple sky decoration, and clear comments.



- The HUD now shows both current score and high score.
- The handwritten note at the bottom explains the endless gameplay loop: keep collecting stars until the player hits an enemy.
- This is the final refinement stage that matches the submitted version of the game.

Problem

The original version ended after reaching a score target, which made the game feel short. Once I tested it more, I realized the gameplay would be more interesting if it continued indefinitely and encouraged replay through a high score system.

Problem-Solving Process

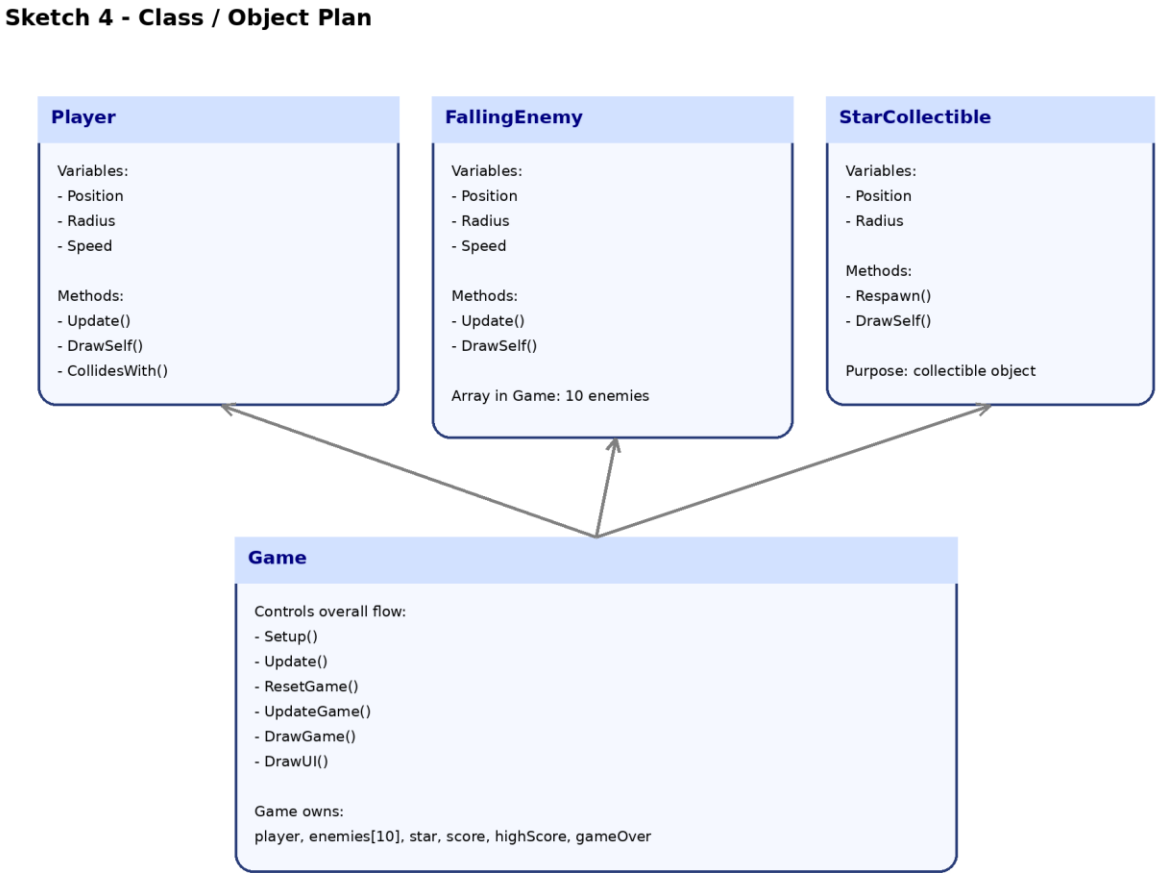
I reviewed how the win condition affected the game loop and saw that the fixed target score stopped the gameplay too early. I decided to remove the target score and win state, then replace them with a high score system. I kept the lose condition because colliding with an enemy still worked well as the failure state. I added a high Score variable and updated it whenever the player beat their previous best score. This made the game feel more repayable without changing the overall structure too much.

Final Decision

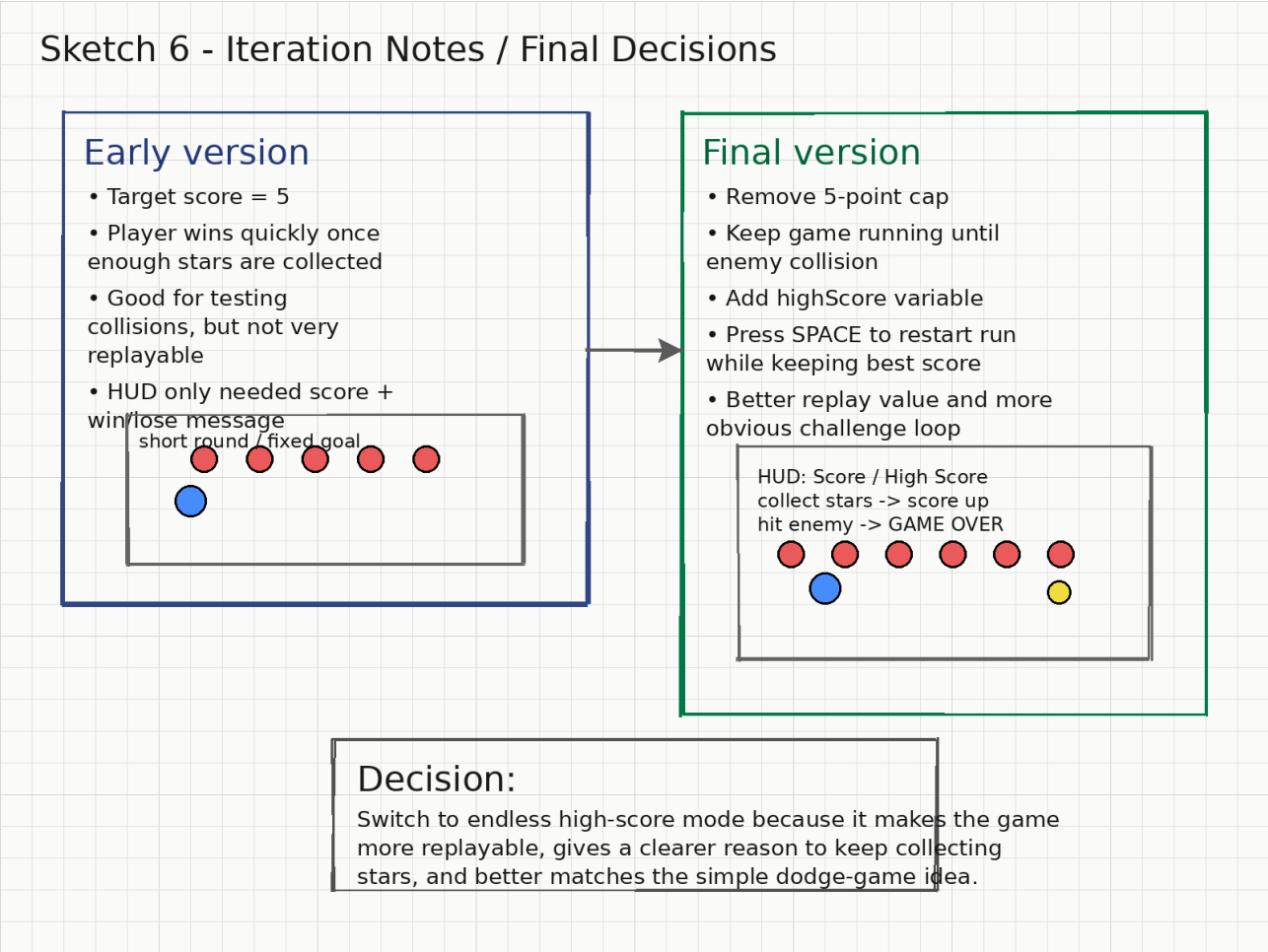
I changed the project to an endless high-score game because it improved replay value and better fit the dodge-and-collect gameplay.

8. Sketch 4 - Class and object planning diagram

This planning page was kept from the earlier process-work draft because it directly shows how the code structure was organized before final implementation. It maps the game controller class to the three user-defined classes and makes the array requirement visible.



9. Iterations



The project started as a short win-condition game, but that made each run end too quickly. After testing the gameplay idea on paper, I changed it to endless scoring with a saved high score. This made the game more repayable, gave the player a reason to keep collecting stars, and matched the simple dodge-game concept better.

10. Small problems I ran into

Input issue

At first, movement felt too fast when both WASD and arrow keys were pressed together. I kept both control options because accessibility was more important, even though this meant players should normally use one control scheme at a time.

Enemy reset issue

Some enemies reset in ways that felt repetitive. I solved this by randomizing the X position when they respawned, while still keeping the initial layout organized for testing.

Collectible placement issue

Sometimes the collectible could appear in awkward positions. I fixed this by limiting its random Y range so it stayed in the main play area and did not appear too close to the ground or top edge.

UI issue

The original UI did not explain the controls well enough. I added instruction text near the top-left corner so the player could understand the game immediately.

Scope issue

I originally considered adding more features like enemy speed scaling or multiple collectible types, but I decided not to because the current version already satisfied the rubric and was more stable.

11. Reflection on the Final Version

The biggest design change was replacing the 5-point win state with endless high-score play. That choice made the project feel more like a repayable arcade game instead of a very short demo. It also made the HUD more meaningful, because the player now tracks both the current run and the best score.

Using circles for the player, enemies, and star kept the visuals simple and matched the collision math, which made both coding and debugging easier. This helped the game stay within the MohawkGame2D template functions while still looking clear and readable on screen.

The sketches helped turn the project into separate, manageable tasks: screen layout first, then class planning, then collision and scoring, then final polish. Matching sketches to commits also made the development process easier to explain and showed visible evidence of iteration from idea to final implementation.